МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №4

з дисципліни **"Функціональне програмування"**

на тему **"РЕКУРСІЯ ТА МЕМОІЗАЦІЯ"**

Варіант 13. Паліндроми

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: зрозуміти структуру рекурсивних обчислень (базовий випадок і крок), обмеження стеку в Python і відсутність TCO, навчитися мемоізувати повторні підзадачі через `functools.lru_cache`; реалізувати рекурсивну роботу з паліндромами (варіант 13), а також quicksort, обходи дерев і мемоізований Фібоначчі.

Теоретичні відомості:

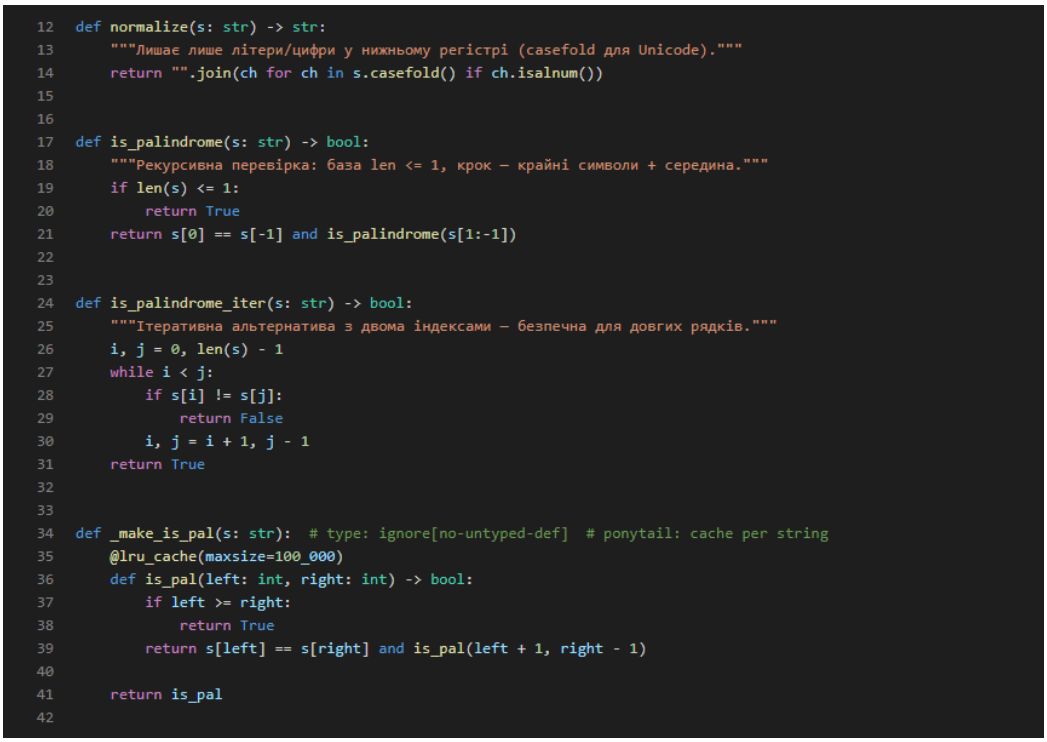
Рекурсивна функція викликає себе над меншими підзадачами. Її обов'язкові елементи — базовий випадок (простий вхід, для якого відповідь відома) і рекурсивний крок (зведення задачі до меншої). Кожен виклик займає кадр стеку; у CPython ліміт глибини за замовчуванням 1000 (`sys.getrecursionlimit()`), при перевищенні виникає `RecursionError`. Хвостова рекурсія не оптимізується (TCO відсутня за задумом розробників), тому для глибоких даних слід переходити до ітерації з власним стеком або чергою.

Мемоізація зберігає результати для вже бачених аргументів. Якщо рекурсія багаторазово обчислює ті самі підзадачі (Фібоначчі, рекурсія по межах рядка), `@lru_cache` перетворює експоненційний час на поліноміальний, а `cache_info()` і `cache_clear()` дозволяють контролювати кеш у тестах. Мемоізація працює лише для чистих функцій із хешованими аргументами.

Хід роботи:

1. За варіантом 13 у `palindromes.py` реалізував `normalize` (casefold та відбір літер і цифр для Unicode-рядків), рекурсивну перевірку `is_palindrome` (база — довжина не більше 1, крок — рівність крайніх символів і паліндромність

середини), ітеративну альтернативу з двома індексами та фабрику кешованого предиката `is_pal(left, right)`:



```python
def normalize(s: str) -> str:
    """Лишає лише літери/цифри у нижньому регістрі (casefold для Unicode)."""
    return "".join(ch for ch in s.casefold() if ch.isalnum())


def is_palindrome(s: str) -> bool:
    """Рекурсивна перевірка: база len <= 1, крок — крайні символи + середина."""
    if len(s) <= 1:
        return True
    return s[0] == s[-1] and is_palindrome(s[1:-1])


def is_palindrome_iter(s: str) -> bool:
    """Ітеративна альтернатива з двома індексами — безпечна для довгих рядків."""
    i, j = 0, len(s) - 1
    while i < j:
        if s[i] != s[j]:
            return False
        i, j = i + 1, j - 1
    return True


def _make_is_pal(s: str):  # type: ignore[no-untyped-def]  # ponytail: cache per string
    @lru_cache(maxsize=100_000)
    def is_pal(left: int, right: int) -> bool:
        if left >= right:
            return True
        return s[left] == s[right] and is_pal(left + 1, right - 1)

    return is_pal

```

2. Реалізував пошук найдовшого паліндромного фрагмента: `longest_palindrome` перебирає межі від найдовших і використовує кешований предикат, а `longest_len` — рекурсія по межах `(left, right)` з `@lru_cache`: якщо фрагмент

паліндром — його довжина, інакше максимум із відкинутим лівим або правим символом. Поруч — та сама рекурсія без кешу для порівняння часу:

```python
def longest_palindrome(s: str) -> str:
    """Найдовший паліндромний фрагмент: рекурсія по межах (left, right) з lru_cache.

    Підзадача is_pal(l, r) кешується, тому кожна пара меж обчислюється один раз
    (O(n²) замість O(n³) для наївного перебору).
    """
    if not s:
        return ""
    is_pal = _make_is_pal(s)
    for length in range(len(s), 0, -1):
        for left in range(0, len(s) - length + 1):
            right = left + length - 1
            if is_pal(left, right):
                return s[left : right + 1]
    return s[0]


@lru_cache(maxsize=100_000)
def longest_len(s: str, left: int, right: int) -> int:
    """Довжина найдовшого паліндромного фрагмента в s[left..right] (з кешем).

    Якщо весь фрагмент паліндром — його довжина; інакше максимум з відкинутим
    лівим або правим символом. Стан (s, left, right) мемоізується.
    """
    if left > right:
        return 0
    if is_palindrome_iter(s[left : right + 1]):
        return right - left + 1
    return max(longest_len(s, left + 1, right), longest_len(s, left, right - 1))


def longest_len_naive(s: str, left: int, right: int) -> int:
    """Та сама рекурсія без кешу — експоненційна, для порівняння часу."""
    if left > right:
        return 0
    if is_palindrome_iter(s[left : right + 1]):
        return right - left + 1
    return max(
        longest_len_naive(s, left + 1, right), longest_len_naive(s, left, right - 1)
    )

```

3. Додав ітеративну альтернативу `longest_palindrome_iter` (розширення від центру без рекурсії) — вона безпечна для довгих рядків, де рекурсія впирається у ліміт стеку:

```python
def longest_palindrome_iter(s: str) -> str:
    """Ітеративна альтернатива: розширення від центру, O(n²), без рекурсії."""
    best = ""
    for center in range(len(s)):
        for left, right in ((center, center), (center, center + 1)):
            while left >= 0 and right < len(s) and s[left] == s[right]:
                left, right = left - 1, right + 1
            if right - left - 1 > len(best):
                best = s[left + 1 : right]
    return best
```

4. У `algorithms.py` виконав базове завдання: чистий рекурсивний `quicksort` з випадковим pivot (повертає новий список), незмінний вузол `Node` і рекурсивні

генератори обходів `preorder`, `inorder`, `postorder`:

```python
def quicksort(xs: list[int]) -> list[int]:
    """Рекурсивний quicksort із випадковим pivot; повертає новий список."""
    if len(xs) <= 1:
        return xs[:]
    p = choice(xs)
    left = [x for x in xs if x < p]
    mid = [x for x in xs if x == p]
    right = [x for x in xs if x > p]
    return quicksort(left) + mid + quicksort(right)


@dataclass(frozen=True, slots=True)
class Node:
    """Вузол бінарного дерева."""

    key: int
    left: Node | None = None
    right: Node | None = None


def preorder(t: Node | None) -> Iterator[int]:
    """Корінь → ліве → праве (рекурсивний генератор)."""
    if t is None:
        return
    yield t.key
    yield from preorder(t.left)
    yield from preorder(t.right)


def inorder(t: Node | None) -> Iterator[int]:
    """Ліве → корінь → праве."""
    if t is None:
        return
    yield from inorder(t.left)
    yield t.key
    yield from inorder(t.right)


def postorder(t: Node | None) -> Iterator[int]:
    """Ліве → праве → корінь."""
    if t is None:
        return
    yield from postorder(t.left)
    yield from postorder(t.right)
    yield t.key

```

5. Додав ітеративні обходи `bfs_level_order` (черга `deque`) і `dfs_preorder_iter` (власний стек), які не залежать від глибини рекурсії, та рекурсивну висоту

дерева:

```python
def bfs_level_order(t: Node | None) -> list[int]:
    """Ітеративний обхід у ширину через deque — не залежить від глибини стеку."""
    if t is None:
        return []
    q, out = deque([t]), []
    while q:
        n = q.popleft()
        out.append(n.key)
        if n.left:
            q.append(n.left)
        if n.right:
            q.append(n.right)
    return out


def dfs_preorder_iter(t: Node | None) -> list[int]:
    """Ітеративний preorder із власним стеком (правий кладемо першим)."""
    if t is None:
        return []
    stack, out = [t], []
    while stack:
        n = stack.pop()
        out.append(n.key)
        if n.right:
            stack.append(n.right)
        if n.left:
            stack.append(n.left)
    return out


def height(t: Node | None) -> int:
    """Висота дерева рекурсивно (порожнє дерево → 0)."""
    return 0 if t is None else 1 + max(height(t.left), height(t.right))

```

6. Реалізував `fib` з `@lru_cache`, наївний `fib_naive` без кешу та ітеративний `fib_iter` з O(1) пам'яті:

```python
@lru_cache(maxsize=100_000)
def fib(n: int) -> int:
    """Мемоізований Фібоначчі: O(n) замість експоненти."""
    if n < 0:
        raise ValueError("n must be >= 0")
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)


def fib_naive(n: int) -> int:
    """Той самий Фібоначчі без кешу — для порівняння часу."""
    return n if n < 2 else fib_naive(n - 1) + fib_naive(n - 2)


def fib_iter(n: int) -> int:
    """Ітеративний Фібоначчі: O(n) часу, O(1) пам'яті, без стеку."""
    if n < 0:
        raise ValueError("n must be >= 0")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

7. Демонстраційний `main.py` перевіряє Unicode-рядки, знаходить найдовший паліндром двома способами, вимірює час `longest_len` і `fib` з кешем і без,

друкує `cache_info()` та ліміт рекурсії:

```python

def main() -> None:
    """Друкує результати та таблицю часу (лише I/O, обчислення чисті)."""
    for s in ["А роза упала на лапу Азора", "Never odd or even", "Привіт"]:
        print(f"{s!r:35} -> {is_palindrome(normalize(s))}")
    text = "abacdfgdcabaxyzzyxq"
    print("longest:", longest_palindrome(text), longest_palindrome_iter(text))

    s = "abcdefghijabcdefghij"
    n = len(s) - 1

    def cached_run() -> int:
        longest_len.cache_clear()
        return longest_len(s, 0, n)

    t_cached = timeit.timeit(cached_run, number=1)
    t_naive = timeit.timeit(lambda: longest_len_naive(s, 0, n), number=1)
    print(f"longest_len({len(s)}) cached: {t_cached * 1e3:.2f} ms")
    print(f"longest_len({len(s)}) naive : {t_naive * 1e3:.2f} ms")
    print("cache_info:", longest_len.cache_info())

    fib.cache_clear()
    print("fib(30) cached:", f"{timeit.timeit(lambda: fib(30), number=1) * 1e3:.3f} ms")
    print(
        "fib(30) naive :",
        f"{timeit.timeit(lambda: fib_naive(30), number=1) * 1e3:.1f} ms",
    )
    print("fib(90) ==", fib(90), "iter:", fib_iter(90))

    print("quicksort:", quicksort([3, 1, 4, 1, 5, 9, 2, 6]))
    t = Node(4, Node(2, Node(1), Node(3)), Node(6, None, Node(7)))
    print("inorder:", list(inorder(t)), "bfs:", bfs_level_order(t))
    print("recursion limit:", sys.getrecursionlimit())


if __name__ == "__main__":
    main()
```

8. Написав тести паліндромів у `tests/test_palindromes.py`: граничні випадки (порожній рядок, один і два символи), Unicode і нормалізація, відомі найдовші

паліндроми, API кешу `cache_info`/`cache_clear`:

```python
@pytest.mark.parametrize(
    ("s", "expected"),
    [
        ("", True),
        ("a", True),
        ("aa", True),
        ("ab", False),
        ("abba", True),
        ("abcba", True),
    ],
)
def test_is_palindrome_edges(s: str, expected: bool) -> None:
    assert is_palindrome(s) is expected
    assert is_palindrome_iter(s) is expected


def test_unicode_and_normalize() -> None:
    assert is_palindrome(normalize("А роза упала на лапу Азора"))
    assert is_palindrome(normalize("Я — ґаздиня, я!")) is False
    assert is_palindrome("шалаш") and is_palindrome("їаїї") and is_palindrome("ABA")
    assert normalize("Ab, c!") == "abc"


def test_longest_palindrome_known() -> None:
    assert longest_palindrome("babad") in {"bab", "aba"}
    assert longest_palindrome("cbbd") == "bb"
    assert longest_palindrome("abacdfgdcabaxyzzyxq") == "xyzzyx"
    assert longest_palindrome("") == ""
    assert longest_palindrome("x") == "x"
    assert longest_palindrome("абвба") == "абвба"


def test_longest_len_cache_api() -> None:
    longest_len.cache_clear()
    s = "abcbaxyz"
    assert longest_len(s, 0, len(s) - 1) == 5
    info = longest_len.cache_info()
    assert info.misses > 0
    longest_len(s, 0, len(s) - 1)
    assert longest_len.cache_info().hits == info.hits + 1
    longest_len.cache_clear()
    assert longest_len.cache_info().currsize == 0

```

9. Додав тест, що кешована версія збігається з наївною і швидша за неї, тест немутації вхідного рядка та property-тести на Hypothesis (рекурсивна версія

дорівнює ітеративній; рядок, склеєний зі своїм відображенням, завжди паліндром):

```python
def test_cached_matches_naive_and_faster() -> None:
    s = "abcdefghijabcdefgh"
    n = len(s) - 1
    longest_len.cache_clear()
    assert longest_len(s, 0, n) == longest_len_naive(s, 0, n) == 1
    t_naive = timeit.timeit(lambda: longest_len_naive(s, 0, n), number=1)

    def cached_run() -> int:
        longest_len.cache_clear()
        return longest_len(s, 0, n)

    t_cached = timeit.timeit(cached_run, number=1)
    assert t_cached < t_naive


def test_input_not_mutated() -> None:
    s = "racecar"
    longest_palindrome(s)
    is_palindrome(s)
    assert s == "racecar"


@given(st.text(max_size=40))
def test_property_recursive_equals_iterative(s: str) -> None:
    assert is_palindrome(s) == is_palindrome_iter(s)
    assert len(longest_palindrome(s)) == len(longest_palindrome_iter(s))
    assert len(longest_palindrome(s)) == longest_len(s, 0, len(s) - 1) if s else True


@given(st.text(max_size=20))
def test_property_doubled_string_is_palindrome(s: str) -> None:
    assert is_palindrome(s + s[::-1])
    assert len(longest_palindrome(s + s[::-1])) >= 2 * len(s)
```

10. Тести базового завдання з методички: quicksort проти `sorted` (включно з property-тестом), послідовності обходів дерева і глибоке лінійне дерево, Фібоначчі з перевіркою кешу та відхиленням від'ємних аргументів:

```python
def test_fib_small() -> None:
    expected = [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
    assert [fib(i) for i in range(10)] == expected
    assert [fib_naive(i) for i in range(10)] == expected


def test_fib_iter_matches() -> None:
    for n in range(50):
        assert fib(n) == fib_iter(n)


def test_cache_clear_and_info() -> None:
    fib.cache_clear()
    assert fib(0) == 0
    fib(25)
    info = fib.cache_info()
    assert info.currsize == 26
    assert info.hits > 0


def test_negative_rejected() -> None:
    with pytest.raises(ValueError):
        fib(-1)
    with pytest.raises(ValueError):
        fib_iter(-1)
```

```python
def sample_tree() -> Node:
    #      4
    #     / \
    #    2   6
    #   / \   \
    #  1   3   7
    return Node(4, left=Node(2, Node(1), Node(3)), right=Node(6, None, Node(7)))


def test_traversals() -> None:
    t = sample_tree()
    assert list(preorder(t)) == [4, 2, 1, 3, 6, 7]
    assert list(inorder(t)) == [1, 2, 3, 4, 6, 7]
    assert list(postorder(t)) == [1, 3, 2, 7, 6, 4]
    assert bfs_level_order(t) == [4, 2, 6, 1, 3, 7]
    assert dfs_preorder_iter(t) == [4, 2, 1, 3, 6, 7]


```

11. Запустив демонстрацію: мемоізована `longest_len` на рядку з 20 символів виконується за 0.2 мс проти 160 мс без кешу (210 станів, 171 попадання), `fib(30)` — 0.03 мс проти 90 мс; ліміт рекурсії — 1000:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab04> python main.py
'А роза упала на лапу Азора'        -> True
'Never odd or even'                 -> True
'Привіт'                            -> False
longest: xyzzyx xyzzyx
longest_len(20) cached: 0.09 ms
longest_len(20) naive : 156.49 ms
cache_info: CacheInfo(hits=171, misses=210, maxsize=100000, currsize=210)
fib(30) cached: 0.034 ms
fib(30) naive : 98.2 ms
fib(90) == 2880067194370816120 iter: 2880067194370816120
quicksort: [1, 1, 2, 3, 4, 5, 6, 9]
inorder: [1, 2, 3, 4, 6, 7] bfs: [4, 2, 6, 1, 3, 7]
recursion limit: 1000
```

12. Перевірив проєкт автоматичними інструментами: усі 24 тести pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab04> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 24 items

tests\test_fib.py::test_fib_small PASSED                                 [  4%]
tests\test_fib.py::test_fib_iter_matches PASSED                          [  8%]
tests\test_fib.py::test_cache_clear_and_info PASSED                      [ 12%]
tests\test_fib.py::test_negative_rejected PASSED                         [ 16%]
tests\test_palindromes.py::test_is_palindrome_edges[-True] PASSED        [ 20%]
tests\test_palindromes.py::test_is_palindrome_edges[a-True] PASSED       [ 25%]
tests\test_palindromes.py::test_is_palindrome_edges[aa-True] PASSED      [ 29%]
tests\test_palindromes.py::test_is_palindrome_edges[ab-False] PASSED     [ 33%]
tests\test_palindromes.py::test_is_palindrome_edges[abba-True] PASSED    [ 37%]
tests\test_palindromes.py::test_is_palindrome_edges[abcba-True] PASSED   [ 41%]
tests\test_palindromes.py::test_unicode_and_normalize PASSED             [ 45%]
tests\test_palindromes.py::test_longest_palindrome_known PASSED          [ 50%]
tests\test_palindromes.py::test_longest_len_cache_api PASSED             [ 54%]
tests\test_palindromes.py::test_cached_matches_naive_and_faster PASSED   [ 58%]
tests\test_palindromes.py::test_input_not_mutated PASSED                 [ 62%]
tests\test_palindromes.py::test_property_recursive_equals_iterative PASSED [ 66%]
tests\test_palindromes.py::test_property_doubled_string_is_palindrome PASSED [ 70%]
tests\test_quicksort.py::test_quicksort_basic PASSED                     [ 75%]
tests\test_quicksort.py::test_quicksort_random PASSED                    [ 79%]
tests\test_quicksort.py::test_quicksort_does_not_mutate PASSED           [ 83%]
tests\test_quicksort.py::test_property_matches_sorted PASSED             [ 87%]
tests\test_tree.py::test_traversals PASSED                               [ 91%]
tests\test_tree.py::test_empty_and_height PASSED                         [ 95%]
tests\test_tree.py::test_iterative_matches_recursive_on_deep_tree PASSED [100%]

============================== 24 passed in 1.41s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab04> python -m mypy .
Success: no issues found in 8 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab04> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab04> python -m black --check .
All done! ✨ 🍰 ✨
8 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я реалізував рекурсивні алгоритми з чітко сформульованими базовими випадками: перевірку паліндрома та пошук найдовшого паліндромного фрагмента за варіантом 13, quicksort, обходи бінарного дерева і Фібоначчі. Мемоізація через `@lru_cache` станів `(left, right)` і значень `n` перетворила експоненційні обчислення на поліноміальні, що підтверджено вимірюванням часу і `cache_info()`. Для кожної задачі є ітеративна альтернатива, яка безпечніша за рекурсію на глибоких даних, оскільки CPython обмежує стек 1000 кадрами і не виконує оптимізацію хвостових викликів. Усі функції чисті й не змінюють вхідні дані, що підтверджено юніт- і property-тестами, mypy, ruff і black.